KPR Institute of Engineering and Technology
(Autonomous)
Avinashi Road, Arasur, Coimbatore - 641 407

LEARN BEYOND

DEPARTMENT OF

COMPUTER SCIENCE AND ENGINEERING

LABORATORY RECORD

U25CSG13 - Object Oriented Programming Using Java Laboratory

Academic Year 2026-27

KPRIET
Learn Beyond

KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY

ARASUR, COIMBATORE – 641407

DEPARTMENT OF COMPUER SCIENCE & ENGINEERING

Name :……

Roll number :……

Year & Semester :……

Class & Section :……

Course Code :……

Course Title :……

Certified that this is a bonafide work done by the above student during the academic year 2026 – 2027.

Faculty in-charge **HoD**

He/ She has submitted the record for the End Semester Practical Examination held on…….

INTERNAL EXAMINER **EXTERNAL EXAMINER**

LIST OF EXPERIMENTS

ACADEMIC YEAR: 2026-2027 (ODD)

Course Code: U25CSG13 **SEMESTER: III**

Course Name: Object Oriented Programming Using Java Laboratory

S No	Date	Name of the Experiment	Marks (20)	Signature
1		Design and Implementation of Java Programs Demonstrating Classes, Objects, Constructors, and Methods		
2		Development of Applications Illustrating Static and Non-static Members, Access Modifiers, Abstraction, and Encapsulation		
3		Implementation of Console-based Applications Using Scanner and System Classes for Input and Output Processing		
4		Design and Implementation of Applications Demonstrating Inheritance and Its Various Types		
5		Development of Applications Illustrating Compile-time and Runtime Polymorphism Using Method Overloading and Method Overriding		
6		Implementation of Applications Using Abstract Classes, Interfaces, and Dynamic Method Dispatch Mechanisms		
7		Design and Development of Modular Java Applications Using Built-in and User-defined Packages		
8		Implementation of Robust Exception Handling Mechanisms Using Built-in and User-defined Exceptions		
9		Development of Multithreaded Applications Using Thread Class and Runnable Interface		
10		Design and Implementation of Synchronized Multithreaded Applications with Inter-thread Communication		
11		Development of Applications Demonstrating Thread Lifecycle Management and Deadlock Prevention Techniques		
12		Implementation of Generic Collection-based Applications Using List, Set, Map, and Queue Interfaces		
13		Design of Applications Utilizing Comparator and Comparable Interfaces for Customized Sorting Operations		
14		Development of File Processing Applications Using Java I/O Streams, Readers, Writers, and Buffered Streams		
15		Implementation of Object Persistence Mechanisms Using Serialization and Deserialization Techniques		

EX.NO: 01	Design and implementation of Java programs demonstrating classes, objects, constructors, and methods
DATE:	

Aim

To design and implement a Java program for a Library Management System using classes, objects, constructors, and methods.

Algorithm / Procedure

Step 1: Start the program.

Step 2: Create a class named Book.

Step 3: Declare the instance variables bookId, title, author, category, price, and available.

Step 4: Create a parameterized constructor to initialize all the instance variables.

Step 5: Create a method named displayBookDetails() to display the book information.

Step 6: Create the Main class containing the main() method.

Step 7: Create two Book objects by passing the required values to the constructor.

Step 8: Invoke the displayBookDetails() method using both objects.

Step 9: Display the details of the books.

Step 10: Stop the program.

Program

Output

DEPARTMENT OF CSE		
Preparation	4	
Observation	4	
Results	4	
Interpretation	4	
Viva-Voce	4	
Total	20	

Result

Thus, the Java program demonstrating the concepts of classes, objects, constructors, and methods through a Library Management System was successfully designed, implemented, and verified with the expected output.

EX.NO: 02	Development of applications illustrating static and non-static members, access modifiers, abstraction, and encapsulation
DATE:	

Aim

To design and implement a Java program for an Employee Management System using static and non-static members, access modifiers, abstraction, and encapsulation.

Algorithm / Procedure

Step 1: Start the program.

Step 2: Create an abstract class named Employee.

Step 3: Declare a static variable company to represent the common company name.

Step 4: Declare non-static variables for employee details using different access modifiers:

public – department

private – salary

protected – designation

package-private (default) – location

Step 5: Create a parameterized constructor to initialize the employee details.

Step 6: Implement getter and setter methods to access and modify the private variable salary, demonstrating encapsulation.

Step 7: Declare an abstract method displayRole() in the abstract class.

Step 8: Create a subclass Developer that extends the Employee class and provides the implementation for the abstract method.

Step 9: Create an object of the Developer class and invoke the methods to display the employee details and role.

Step 10: Stop the program.

Program

Output

DEPARTMENT OF CSE		
Preparation	4	
Observation	4	
Results	4	
Interpretation	4	
Viva-Voce	4	
Total	20	

Result

Thus, the Java program for an Employee Management System demonstrating static and non-static members, access modifiers, abstraction, and encapsulation was successfully designed, implemented, and verified with the expected output.

EX.NO: 03	Implementation of console-based applications using Scanner and System classes for input and output processing
DATE:	

Aim

To design and implement a Java program for a Student Admission Management System using the Scanner and System classes for input and output processing.

Algorithm / Procedure

Step 1: Start the program.

Step 2: Import the java.util.Scanner package.

Step 3: Create a Scanner object to receive input from the user.

Step 4: Read the student details such as Student ID, Student Name, Department, Age, and Percentage using the Scanner class.

Step 5: Display the entered student details using the System.out.println() method.

Step 6: Close the Scanner object.

Step 7: Stop the program.

Program

Output

DEPARTMENT OF CSE		
Preparation	4	
Observation	4	
Results	4	
Interpretation	4	
Viva-Voce	4	
Total	20	

Result

Thus, the Java program for a Student Admission Management System using the Scanner and System classes for input and output processing was successfully designed, implemented, and verified with the expected output.

EX.NO: 04	Design and implementation of applications demonstrating inheritance and its various types
DATE:	

Aim

To design and implement a Java program for a Banking System demonstrating the various types of inheritance in Java.

Algorithm / Procedure

Step 1: Start the program.

Step 2: Create a parent class named Account containing the account holder's name and account number.

Step 3: Create a child class SavingsAccount that inherits the properties of the Account class.

Step 4: Create another child class CurrentAccount that also inherits the properties of the Account class.

Step 5: Create a class PremiumSavingsAccount that extends SavingsAccount to demonstrate multilevel inheritance.

Step 6: Create objects for SavingsAccount, CurrentAccount, and PremiumSavingsAccount.

Step 7: Invoke the inherited methods to display the account details.

Step 8: Display the output.

Step 9: Stop the program.

Program

Output

DEPARTMENT OF CSE		
Preparation	4	
Observation	4	
Results	4	
Interpretation	4	
Viva-Voce	4	
Total	20	

Result

Thus, the Java program for a Banking System demonstrating single, multilevel, and hierarchical inheritance was successfully designed, implemented, and verified with the expected output.

EX.NO: 05	Development of applications illustrating compile-time and runtime polymorphism using method overloading and method overriding
DATE:	

Aim

To design and implement a Java program for an Online Payment System demonstrating compile-time and runtime polymorphism using method overloading and method overriding.

Algorithm / Procedure

Step 1: Start the program.

Step 2: Create a class named Payment.

Step 3: Implement method overloading by creating multiple makePayment() methods with different parameter lists.

Step 4: Create a subclass UPIPayment that extends the Payment class.

Step 5: Override the makePayment(double amount) method in the subclass to demonstrate runtime polymorphism.

Step 6: Create an object of the UPIPayment class.

Step 7: Invoke the overloaded methods to demonstrate compile-time polymorphism.

Step 8: Invoke the overridden method using the subclass object to demonstrate runtime polymorphism.

Step 9: Display the payment details.

Step 10: Stop the program.

Program

Output

DEPARTMENT OF CSE		
Preparation	4	
Observation	4	
Results	4	
Interpretation	4	
Viva-Voce	4	
Total	20	

Result

Thus, the Java program for an Online Payment System demonstrating compile-time and runtime polymorphism using method overloading and method overriding was successfully designed, implemented, and verified with the expected output.

EX.NO: 06	Implementation of applications using abstract classes, interfaces, and dynamic method dispatch mechanisms
DATE:	

Aim

To design and implement a Java program for a Smart Home Appliance System using abstract classes, interfaces, and dynamic method dispatch.

Algorithm / Procedure

Step 1: Start the program.

Step 2: Create an interface named RemoteControl with the methods turnOn() and turnOff().

Step 3: Create an abstract class named Appliance containing an abstract method displayAppliance().

Step 4: Create a class SmartTV that extends the Appliance class and implements the RemoteControl interface.

Step 5: Provide implementations for all the abstract and interface methods.

Step 6: Create a parent class reference of type Appliance and assign it to a SmartTV object to demonstrate dynamic method dispatch.

Step 7: Invoke the methods using the parent class reference.

Step 8: Display the appliance details and status.

Step 9: Stop the program.

Program

Output

DEPARTMENT OF CSE		
Preparation	4	
Observation	4	
Results	4	
Interpretation	4	
Viva-Voce	4	
Total	20	

Result

Thus, the Java program for a Smart Home Appliance System using abstract classes, interfaces, and dynamic method dispatch was successfully designed, implemented, and verified with the expected output.

EX.NO: 07	Design and Development of Modular Java Applications Using Built-in and User-defined Packages
DATE:	

Aim

To design and develop a modular Java application using built-in and user-defined packages for organizing classes and promoting code reusability.

Algorithm

1. Start the program.
2. Import the Employee class from the employee package and the Scanner class.
3. Create a Scanner object to read input from the user.
4. Read the employee ID from the user.
5. Read the employee name from the user.
6. Read the employee salary from the user.
7. Create an Employee object by passing the employee ID, name, and salary to the constructor.
8. Invoke the displayEmployee() method to display the employee details.
9. Close the Scanner object.
10. Stop the program.

Program

Output

DEPARTMENT OF CSE		
Preparation	4	
Observation	4	
Results	4	
Interpretation	4	
Viva-Voce	4	
Total	20	

Result

Thus, a modular Java application using built-in and user-defined packages was designed and implemented successfully.

EX.NO: 08	Implementation of Robust Exception Handling Mechanisms Using Built-in and User-defined Exceptions
DATE:	

Aim

To implement robust exception handling mechanisms in Java using built-in and user-defined exceptions.

Algorithm

1. Start the program.
2. Define a user-defined exception class InsufficientBalanceException.
3. Create the BankAccount class with a balance attribute and a constructor to initialize it.
4. Define the withdraw() method to check whether the withdrawal amount exceeds the available balance.
5. If the withdrawal amount is greater than the balance, throw an InsufficientBalanceException.
6. Otherwise, deduct the withdrawal amount from the balance and display the updated balance.
7. In the main() method, create a Scanner object to read user input.
8. Read the initial account balance from the user.
9. Create a BankAccount object with the entered balance.
10. Read the withdrawal amount from the user.
11. Invoke the withdraw() method.
12. Handle InputMismatchException if the user enters invalid (non-numeric) input.
13. Handle InsufficientBalanceException if the withdrawal amount exceeds the available balance.
14. Execute the finally block to display the transaction completion message and close the Scanner.
15. Stop the program.

Program

Output:

DEPARTMENT OF CSE		
Preparation	4	
Observation	4	
Results	4	
Interpretation	4	
Viva-Voce	4	
Total	20	

Result

Thus, exception handling using built-in and user-defined exceptions was implemented successfully.

EX.NO: 09	Development of Multithreaded Applications Using Thread Class and Runnable Interface
DATE:	

Aim

To develop multithreaded Java applications using the Thread class and Runnable interface.

Algorithm

1. Start the program.
2. Create a class ReservationThread by extending the Thread class.
3. Override the run() method to display the ticket reservation status five times.
4. Pause the execution for 500 milliseconds after each reservation using Thread.sleep().
5. Handle the InterruptedException, if it occurs.
6. Create a class StatusThread by implementing the Runnable interface.
7. Override the run() method to display the ticket confirmation status five times.
8. Pause the execution for 500 milliseconds after each confirmation using Thread.sleep().
9. Handle the InterruptedException, if it occurs.
10. In the main() method, create an object of ReservationThread.
11. Create a Thread object by passing an object of StatusThread to its constructor.
12. Invoke the start() method for both thread objects to execute them concurrently.
13. Display the reservation and ticket confirmation messages as both threads execute simultaneously.
14. Stop the program.

Program

Output

DEPARTMENT OF CSE		
Preparation	4	
Observation	4	
Results	4	
Interpretation	4	
Viva-Voce	4	
Total	20	

Result

Thus, a multithreaded Java application using the Thread class and Runnable interface was developed successfully.

EX.NO: 10	Design and Implementation of Synchronized Multithreaded Applications with Inter-thread Communication
DATE:	

Aim

To design and implement synchronized multithreaded applications using synchronization and inter-thread communication.

Algorithm

1. Start the program.
2. Create a shared Buffer class with a data variable and an availability flag.
3. Define the produce() method as synchronized to produce data into the buffer.
4. If the buffer already contains data, make the producer thread wait until the consumer consumes it.
5. Store the produced value in the buffer, display it, mark the buffer as available, and notify the waiting consumer thread.
6. Define the consume() method as synchronized to consume data from the buffer.
7. If the buffer is empty, make the consumer thread wait until the producer produces data.
8. Read and display the data from the buffer, mark the buffer as empty, and notify the waiting producer thread.
9. Create the Producer thread class to produce five values and invoke the produce() method repeatedly.
10. Create the Consumer thread class to consume five values and invoke the consume() method repeatedly.
11. Introduce a delay of 500 milliseconds after each produce and consume operation using Thread.sleep().
12. In the main() method, create a shared Buffer object.
13. Create Producer and Consumer thread objects by passing the shared Buffer object.
14. Start both threads using the start() method to execute the producer-consumer process concurrently.

15. Stop the program.

Program

Output

DEPARTMENT OF CSE		
Preparation	4	
Observation	4	
Results	4	
Interpretation	4	
Viva-Voce	4	
Total	20	

Result

Thus, synchronized multithreading with inter-thread communication was implemented successfully.

EX.NO: 11	Development of Applications Demonstrating Thread Lifecycle Management and Deadlock Prevention Techniques
DATE:	

Aim

To develop Java applications demonstrating thread lifecycle management and deadlock prevention techniques.

Algorithm

1. Start the program.
2. Create a shared Printer class with a synchronized printDocument() method.
3. Display the thread name indicating that the document is being printed.
4. Pause the thread for 1000 milliseconds using Thread.sleep() to simulate the printing process.
5. Display a message indicating that the printing operation has finished.
6. Create a UserThread class by extending the Thread class.
7. Pass the shared Printer object and thread name to the UserThread constructor.
8. Override the run() method to display the thread start message, invoke the printDocument() method, and display the thread termination message.
9. In the main() method, create a shared Printer object.
10. Create two UserThread objects using the shared Printer object.
11. Display the initial state of the first thread using the getState() method.
12. Start both threads using the start() method.
13. Pause the main thread briefly and display the current state of the first thread.
14. Wait for both threads to complete execution using the join() method.
15. Display the final state of the first thread and print the deadlock prevention message indicating that synchronization with a single shared lock prevents deadlock.
16. Stop the program.

Program

Output

DEPARTMENT OF CSE		
Preparation	4	
Observation	4	
Results	4	
Interpretation	4	
Viva-Voce	4	
Total	20	

Result

Thus, thread lifecycle management and deadlock prevention techniques were demonstrated successfully.

EX.NO: 12	Implementation of Generic Collection-Based Applications Using List, Set, Map, and Queue Interfaces
DATE:	

Aim

To implement generic collection-based applications using List, Set, Map, and Queue interfaces.

Algorithm

1. Start the program.
2. Create an ArrayList object using the List interface.
3. Add student names to the list, including duplicate values.
4. Display the contents of the student list.
5. Create a HashSet object using the Set interface.
6. Add course names to the set and observe that duplicate values are automatically removed.
7. Display the contents of the course set.
8. Create a HashMap object using the Map interface.
9. Insert student roll numbers as keys and student names as values into the map.
10. Traverse the map using entrySet() and display each roll number with its corresponding student name.
11. Create a LinkedList object using the Queue interface.
12. Insert student names into the queue using the offer() method.
13. Display the queue before processing.
14. Remove the front element from the queue using the poll() method and display the processed element.
15. Display the queue after processing.
16. Stop the program.

Program

Output:

DEPARTMENT OF CSE		
Preparation	4	
Observation	4	
Results	4	
Interpretation	4	
Viva-Voce	4	
Total	20	

Result

Thus, generic collection-based applications using List, Set, Map, and Queue interfaces were implemented successfully.

EX.NO: 13	Design of Applications Utilizing Comparator and Comparable Interfaces for Customized Sorting Operations
DATE:	

Aim

To design Java applications utilizing Comparable and Comparator interfaces for customized sorting operations.

Algorithm

1. Start the program.
2. Create a Student class with rollNo and name as data members.
3. Implement the Comparable interface in the Student class to define the default sorting order based on roll number.
4. Override the compareTo() method to compare students using their roll numbers.
5. Override the toString() method to display the student details.
6. Create a NameComparator class by implementing the Comparator interface.
7. Override the compare() method to compare students based on their names.
8. Create an ArrayList to store Student objects.
9. Add student objects with roll numbers and names to the list.
10. Display the list of students before sorting.
11. Sort the list by roll number using the Collections.sort() method and the Comparable implementation.
12. Display the students sorted by roll number.
13. Sort the list by name using the Collections.sort() method with the NameComparator object.
14. Display the students sorted by name.
15. Stop the program.

Program

Output

DEPARTMENT OF CSE		
Preparation	4	
Observation	4	
Results	4	
Interpretation	4	
Viva-Voce	4	
Total	20	

Result

Thus, customized sorting using Comparable and Comparator interfaces was implemented successfully.

EX.NO: 14	Development of File Processing Applications Using Java I/O Streams, Readers, Writers, and Buffered Streams
DATE:	

Aim

To develop Java applications for file processing using Java I/O streams, readers, writers, and buffered streams.

Algorithm

1. Start the program.
2. Specify the file name to store the student details.
3. Create a FileWriter object to write data to the file.
4. Wrap the FileWriter with a BufferedWriter object for efficient writing.
5. Write the student record and its details to the file line by line.
6. Close the BufferedWriter to save the data to the file.
7. Display a message indicating that the data has been written successfully.
8. Create a FileReader object to read data from the file.
9. Wrap the FileReader with a BufferedReader object for efficient reading.
10. Read the file contents line by line using the readLine() method.
11. Display each line of the file on the console until the end of the file is reached.
12. Close the BufferedReader.
13. Handle any IOException that occurs during file operations.
14. Stop the program.

Program

Output

DEPARTMENT OF CSE		
Preparation	4	
Observation	4	
Results	4	
Interpretation	4	
Viva-Voce	4	
Total	20	

Result

Thus, file processing using Java I/O streams and buffered streams was implemented successfully.

EX.NO: 15	Implementation of Object Persistence Mechanisms Using Serialization and Deserialization Techniques
DATE:	

Aim

To implement object persistence mechanisms using serialization and deserialization techniques in Java.

Algorithm

1. Start the program.
2. Create a Student class that implements the Serializable interface.
3. Declare the student attributes (rollNo, name, and cgpa) and initialize them using a constructor.
4. Define the display() method to display the student details.
5. Create a Student object with the required values.
6. Create a FileOutputStream object to open the serialization file.
7. Create an ObjectOutputStream object and write the Student object to the file using the writeObject() method.
8. Close the ObjectOutputStream and display a message indicating successful serialization.
9. Create a FileInputStream object to open the serialized file.
10. Create an ObjectInputStream object and read the serialized object using the readObject() method.
11. Typecast the returned object to the Student class.
12. Close the ObjectInputStream and display a message indicating successful deserialization.
13. Invoke the display() method to display the deserialized student details.
14. Handle IOException and ClassNotFoundException, if they occur.
15. Stop the program.

Program

Output

DEPARTMENT OF CSE		
Preparation	4	
Observation	4	
Results	4	
Interpretation	4	
Viva-Voce	4	
Total	20	

Result

Thus, object persistence using serialization and deserialization techniques was implemented successfully.